

Best Practices for Machine Readable Data and Code Sharing in Psychology

Daniël Lakens¹, Cristian Mesquida², and Lisa DeBruine³

¹Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

²Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

³Institute of Neuroscience and Psychology, University of Glasgow

Author Note

Daniël Lakens  <https://orcid.org/0000-0002-8393-5316>

Cristian Mesquida  <https://orcid.org/0000-0002-1542-8355>

Lisa DeBruine  <https://orcid.org/0000-0002-7523-5539>

A reproducible version of this manuscript available at on GitHub (https://github.com/scienceverse/machine_readable_FAIR_data_and_code). The authors have no conflict of interest to declare. This work was funded with the support of the Netherlands Organization for Scientific Research (NWO Vici grant VI.C.241.013). Author roles were classified using the Contributor Role Taxonomy (CRediT; <https://credit.niso.org/>) as follows: Daniël Lakens: conceptualization, writing, funding acquisition; Cristian Mesquida: conceptualization, writing; Lisa DeBruine: conceptualization, writing

Correspondence concerning this article should be addressed to Daniël Lakens, Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Den Dolech 2, 5600 MB, Eindhoven, Email: D.Lakens@tue.nl

Abstract

The FAIR principles state that research materials should be Findable, Accessible, Interoperable, and Reusable (?). Best practices for FAIR data and code involve the creation of consistently structured repositories and standardised, machine-readable meta-data, but adhering to these best practices is a complex and effortful task. Here we describe a variety of best practices and existing standards, and introduce software that can facilitate these practices.

Keywords: FAIR, Data sharing, Reproducibility, Open Science

Best Practices for Machine Readable Data and Code Sharing in Psychology

Access to the data and code that underlie findings in the scientific literature allows peers to scrutinize the evidential basis for empirical claims, and to increase the efficiency of knowledge generation through re-use. The [European Code of Conduct for Research Integrity](#) states: “Researchers, research institutions, and organisations ensure that access to data is as open as possible, as closed as necessary, and where appropriate in line with the FAIR Principles (Findable, Accessible, Interoperable and Reusable) for data management.” Similarly, the [Dutch Code of Conduct for Research Integrity](#) states that: “Ensure that, in accordance with the FAIR principles, data is open and accessible to the extent possible and remains confidential to the extent necessary.” Research materials should be Findable, Accessible, Interoperable, and Reusable (?). The FAIR Guidelines were explicitly developed with human and mechanical users in mind. While it is important that any individual can re-use data, the benefits of FAIR data and code will only achieve their full potential if machines can access and process scientific data. Therefore, scientific data and code should be shared so that it is machine readable ([Lakens & DeBruine, 2021](#)). Machine readable FAIR data and code will make it possible to retrieve and integrate data in large searchable databases, and to automate actions on data and code, such as checks for good research practices. This guide to data and code sharing summarizes best practices to make data and code findable, accessible, interoperable, and reusable by machines.

A machine can only find data and code in a reliable and automatic manner if there is a stable digital identifier and metadata, access it standardized protocols (for example through HTTPS, or an Application Programming Interface (API)), the shared content can be used by other software (for example by sharing it in non-proprietary formats, with metadata that describes how a file should be read in), and re-usable (for example by providing a codebook and a license). If data and code are shared well, an autonomously-acting computational algorithm can identify what files are (both their format, and why a file is shared) based on meta-data or the filename, determine if it is usable (for example given the license or privacy regulations), and use the data. Ideally, each step is unambiguous and can be performed deterministically, minimizing the need for probabilistic inference from machine-learning

algorithms or large language models. This guide is written for researchers and data managers. We will discuss where to share data, repository organization, file naming, file formats, codebooks, code, README and requirement files, data layout, column naming, codebook generation, specifying scales and measures, and adhering to existing schemas for each.

Practices

1 Repository Choice

There are many data repositories, each with strengths and weaknesses. When choosing a data repository, you may be interested in the ability to provide anonymized versions of the repository, make some files private, or control access. You will also want to consider whether the repository provides stable and unchangeable versions, persistent identifiers like DOIs, and version control. You may need to consider a repository's compatibility with regulations such as GDPR or funder requirements, and limitations on the type or quantity of data you can archive. Some data repositories commonly used in psychology are Zenodo, GitHub, the Open Science Framework, ResearchBox, Figshare, and PsyArchives. In addition, countries or universities might provide data storage, such as the UK Data Service or DataverseNL. Files on GitHub are not permanent and do not have a stable identifiers, but a repository can easily be shared on Zenodo. ResearchBox and PsychArchives currently do not have a public API, which makes it difficult for a machine to access files. It is often desirable to share data and code anonymously for peer review, which can easily be done on PsychArchives, Dataverse, ResearchBox, and the Open Science Framework, and with some effort on Zenodo and GitHub. Zenodo is often the good general-purpose repository for anonymous data, but as its host CERN is an international organization, it can't legally comply with the European General Data Protection Regulations, which makes it a less appropriate choice for pseudo-anonymized data.

2 Which Files to Share

Data repositories might contain files that researchers share to facilitate *replication* (i.e., materials and stimuli used in the study), and files that facilitate *reuse*, for example to reproduce results. Both types of files should be shared, but they should be shared in different ways. Materials vary considerably between studies, and will rarely be processed by a machine. Data and code contain information that others might want to directly access (e.g., to enable live

meta-analyses, or to search for recent studies that use a similar questionnaire). Where possible, researchers should share raw data (e.g., the format in which data was created, such as an exported qualtrics file, audio recordings, individual participant EEG data).

It is often not possible to share raw data, for example because the raw data is not in digital format, or because the raw data is not pseudo-anonymized. In these situations the primary data (i.e., the first digital data that can be shared) should be included in the repository, such as scanned surveys, or anonymized transcripts. If you need to omit specific columns of the original dataset (e.g., IP addresses or other personally identifying information) or process messy data into a tabular datafile that can be analyzed, use code to create a minimally processed version that adheres to the principles above, share the code file, and explain in the README how the primary data was created from the raw data using the code. Using code, rather than manually editing the raw data, allows the data pre-processing pipeline to be audited for potential mistakes, and documents the ways in which the raw data were altered.

In addition to data, and code, researchers might want to share output files, which can include results of statistical analyses, tables, and figures. Ideally, these output files are computationally reproducible from the data and code, but especially when output takes a long time to reproduce (e.g., the results of simulations) it can be convenient to share them even if they are perfectly reproducible. And finally, researchers might also want to share documentation, such as a codebook, preregistration documents, the manuscript and supplementary materials, metadata files such as a readme.

Online data repositories often offer generous file size limits. For example, Zenodo allows users to share 50GB per record, in at most 100 files. The size of files can create a challenge for (automatic) re-use, as users need sufficient disk space to download the files they want to re-use. Downloading and unzipping a 10GB archive requires more resources than an archive of 1MB, and not all users might have sufficient disk space. When sharing files, it is important to keep repositories as small as possible, and to share files in a way that they can be selectively downloaded. Automated tools will need to apply limits to the size of files and repositories that are downloaded. When a repository size is larger than what can be downloaded, tools will likely download files up to the file size limit, download only parts

of data (i.e., through a range request), and prioritize the download of certain files (for example, ignore individual level data files, and only download the merged dataset used to run the analyses reported in the manuscript).

If .csv files are very large, the file size can be substantially reduced by compressing them into a .gzip format. As .gzip files can still be read in directly without needing to unzip the archive, this file format should be used for .csv files larger than approximately 10 MB to reduce size of the repository. For other large files it can be beneficial to compress them as a .zip archive. A .zip archive stores its file index (the “central directory”) in the tail of the archive, so code can fetch only the last 128 KB of a .zip archive via an HTTP range request to read the name and size of every file inside the archive. This makes it possible to determine whether a .zip archive holds data worth downloading. For this reason the recommendation is to only use .zip files as an archive. Due to their proprietary format, .rar archives should not be used.

Clearly label files to facilitate selective downloads. If a users wants to reproduce the data analysis, and a repository contains 14GB of video files used as stimuli, the user should be able to download only the data and code files, which prevents the waste of time and resources downloading unnecessary files. Software that automatically downloads the files you share will need to incorporate file size and repository size limits. For example, a tool that peer reviewers can use to automatically check if the reported results can be reproduced from the data and code can only download files up to disk space the peer reviewer has available. The practice of clearly labeling files in repositories is also important to distinguish links to repositories that contain existing software used in research (e.g., a GitHub link to a software tool created by others that was used to process physiological data) from links to repositories that contain code generated by the researchers themselves for their research (e.g., code for an agent-based model that researchers have created as part of their research).

Some files can be completely reproduced by other files in a repository. For example, code for computer simulations might create a large .csv file with the results of hundreds of thousands of simulations. Labeling such files as ‘output’ will make it possible to download them only when needed. Long-term storage of output comes at cost, so the general recommendation is that outputs should only be stored when they substantially reduce

computational burden or facilitate verification. Computational reproducibility refers to being able to produce the same outputs from the same data and code inputs (?). Computational reproducibility is often threatened by different versions of software or settings. To increase computational reproducibility, software and dependencies should be clearly indicated. For example, R projects can include an `renv.lock` file and Python projects a `requirements.txt` or `environment.yml` file. For analyses that depend on system libraries or software beyond R or Python packages, containerized solutions such as Docker, Nix, or Code Ocean provide stronger guarantees of long-term reproducibility.

3 File and Folder Naming Conventions

As unanimous agreement on naming conventions is difficult to achieve, the folder structure and naming conventions of folders and files will vary between research projects and fields. Nevertheless, some basic principles should be applied to facilitate actions by machines. Folders are used to group related files. For scientific papers, folders indicates projects, such as single studies, and categories of files. Most research files can be sorted in the folders data, code, outputs, materials, and documentation, with an ‘other’ folder for files that do not fit the other five categories. The top level folder is associated with a paper, and it contains files that apply to all sub-folders in the repository. An example of a file that belongs to the top level folder is a README – a text document that provides a human-readable description of the contents of the repository, and instructions on how to reuse the contents. The Psych-DS format, a data structure for research in the social sciences, recommends that repositories should contain a sub-folder for each project in which data was collected. This means that for multi-study manuscripts, the repository contains a sub-folder for each study (e.g., ‘pilot-classification’, ‘study-mood-classification’, ‘study-moderator-effect-mood’). Try to name studies consistently and clearly from the first documentation (e.g., an ethical review application or a preregistration). As studies might not be communicated in the same order as they are performed, and as numbers are not a good description of a study, do not refer to ‘study 1’ but use a short but descriptive label that assists machines to link data in a repository to a specific study in a manuscript. If the final order in which studies is determined (e.g., when a manuscript is written) it is helpful to add a number to each subfolder that contains data from a

study (e.g., ‘01-pilot-classification’, ‘02-study-mood-classification’).

At a minimum, every folder for a single study should contain a ‘data’ folder. Data files need a codebook. Some analysis software (e.g., JASP and Jamovi) store data, codebooks, code, and output in a single file (i.e., a .jasp or .omv file). This removes the need for additional files and folders, as a single .jasp or .omv file in a data folder will allow a machine to retrieve all the information it needs. If all analyses that you want to perform are available in JASP or Jamovi, the easiest solution for machine-readable FAIR data and code sharing is to import the data, enter the codebook information, perform the analyses, and share the .jasp or .omv file. This will most often be an easier and more reproducible solution than performing your analyses in R, where sharing data and code equally FAIR requires substantially more work and expertise. If a standalone codebook needs to be created there are software packages that can assist ([Arslan, 2019](#)). If data is analyzed with individual code files, these are placed in a ‘code’ folder. The study folder might also contain an ‘output’ folder that contains the stored results from the analysis scripts. In addition to a data and code folder, repositories might contain a ‘documentation’ folder (for the preregistration document, the manuscript itself, etc.), and a ‘materials’ folder (for stimuli, survey files, etc.). Software will likely search for each of these folders under a range of aliases (e.g., the documentation folder might be called ‘reports’ or ‘docs’, the materials folder might be called ‘assets’ or ‘stimuli’, etc.). As long as the variation in file names is small, software can search for a list of folder names that contain a specific file type.

Files and folders should be named to be human-readable, and allow classification by machines. As files will often need to be classified on multiple properties (e.g., a unique identifier, and the type of file) a machine should be able to separate these properties in a file name. A good convention is to use an underscore to distinguish different properties in a file name. For example, the file name ‘study-01-classification_data.csv’ allows a machine to identify the type of file (i.e., a file that contains data), and the unique identifier (i.e., study-01). For file classification, the use of underscores and dashes will not matter, and even CamelCase file names (e.g., ‘codePowerSimulationStudy1’) should allow a machine to classify files. What is essential, however, is that each file name includes a word that allows a machine to classify it

into a category. For scientific data, the main data types are data, code, output, documentation, and materials. The file name is an important source of information for machines to classify the possible content of folders, and it should include one of these five words (or a sufficiently similar word, such as `data` `dictionary` `stimuli`). File names should also include a descriptive keyword (e.g., `preprint_documentation.pdf`). Given the lack of unanimous agreement on file naming conventions, machines will need to classify every file in a repository using filename and extension rules, with a large-language-model fallback for files that remain ambiguous after rule-based classification fails (i.e., when files are not named well).

Some software might interpret names with and without a capital as different files (e.g., `Data.csv` versus `data.csv`). Be consistent in the use of capitals. In general, it is recommended to only use lowercase letters, unless schemas and other data standards require capitalized filenames. Do not use spaces or special characters in filenames. Allow for lexicographic sorting of numbered files (e.g., if `'2-data'` precedes `'11-data'` name the files `'02-data'` and `'11-data'`). If you plan to limit the number of files in a repository to 99 (a requirement on Zenodo), two digits should be sufficient (but studies with a large number of stimuli, or individual data files per participant, might require more digits for the files inside a .zip archive). Dates should be provided in YYYYMMDD format following the ISO 8601 basic calendar format (e.g., `'20260724_data.csv'` or `'2026-07-24_data.csv'`).

The use of subfolders can lead to long file and folder names. Certain file systems and software tools limit filenames at 255 characters. Furthermore, a folder needs to be downloaded to an existing folder, which adds to the total length of the filename. For example, if one wants to download a repository to `C:\Users\dlakens\Downloads` the filenames in the repository should remain below 228 characters. Try to keep the directory names (e.g., `'study-1-representative-sample/individual-participant-files/accuracy/'`) below 100 characters, and file names below 50 characters, so that there are around 100 characters left for the local folder name. Names should match the information in the manuscript. If the manuscript refers to `'Study 1'` the folder in the survey should be named `'Study 1'`, and not `'Survey 1'` or `'Aggression Study'`. This allows machines to link data and code in multi-study papers to the

correct section of the manuscript. Meta-data (for example following the RO-Crate standard, as discussed in the next section) should be included to provide further information about the files and folders in a repository.

4 Describe the Content and Organization of the Files

A common recommendation for data and code sharing is to add a README document in every repository. README documents are useful for people who manually browse through the contents of a repository, and should therefore be included to describe the files included in a repository to re-users. However, a README is not very useful for machines. A README should be complemented by a structured counterpart, which includes information that can't easily be retrieved from the files in the repository, while being of general interest to re-users.

This structured meta-data file should provide information about the files in a repository that are difficult to retrieve from the files themselves. Information should be provided about which research project the data belongs to, and who created it. While researchers might think that re-users will find their files through the article they have written, machines are more likely to find repositories directly. Therefore, the meta-data needs to describe the project fully (and link to a DOI of the manuscript if one exists). Although good file naming should allow re-users to know what all files in a repository are, adding this information automatically to the JSON file serves as a validation that file names were unambiguous and provides ground truth when names may be necessarily ambiguous.

Adding meta-data to a repository is complex. There are multiple meta-data standards that operate on different levels. Dublin Core meta-data standards store generic information about the repository (such as the title, description, creator, and license). The [DDI Alliance](#) provides ISO-standard specifications designed to describe research data in the social, demographic, economic, and health sciences. The [DDI-Lifecycle 3.3](#) standard can be used to describe study design, sampling, and data, and supports the reuse of data. Field-specific meta-data standard can be developed to describe files common in specific research areas, such as BIDS for brain data, or Psych-DS for research data in psychology. Finally, there are meta-data standard to describe specific types of research data in specific fields, such as the [Open Scales Database](#) specification for scales in psychology.

There is a clear need for field-specific guidelines on which meta-data should be shared. Furthermore, tools should be developed that make it easy for researchers to generate JSON structured meta-data files that are compliant with widely used meta-data specifications. These recommendations will likely draw on existing standards, such as the [DDI-Lifecycle 3.3](#) for the study, sample, and data-handling descriptions, [CRediT](#) for contributor roles, [schema.org](#) for bibliographic metadata, and the RO-crate specification (?) to document the relationship between files in a repository to support machine readable FAIR principles.

Some information is difficult to retrieve from a paper, but valuable to have available for re-use. Such information can be directly coded into meta-data files. Which information is important enough to be added to meta-data formats is a matter research fields should reach consensus on. In many empirical disciplines, information about the sample size is worth documenting, as the amount of information collected is re-used in meta-analyses, and in part determines the strength of the evidence. The DDI-Lifecycle schema has standardized meta-data for sample eligibility criteria, inclusion and exclusion criteria, recruitment, the target sample size, the number of completed responses, and the resulting response rate. As data provenance (a log of how a study's data came into existence) is becoming increasingly important, it might also be useful to add metadata describing the time window during which data was collected, the situation in which it was collected (for example, whether sessions were supervised, individual or group, or ran during an exam period), and the mode of collection (e.g., an online survey, in person interviews). DDI's LifecycleEvent schema allows researchers to list each study's handling steps as a series of events, stating what happened, when, and who was responsible. This makes it possible to log anyone who handled the data but is not an author. This could be a research assistant who ran the sessions, a technician who exported the files, or a data manager who archived the files. Adding meta-data about these contributions makes this labour visible, and clearly logs who is responsible for each data processing step in the case of suspicions of research integrity violations. As a final example, the increased interest in tracking whether manuscripts adhere to open science practices might deserve the future development of a dedicated meta-data schema for open science practices indicating information such as whether all data that can be shared has been made available, the

code to reproduce all reported results has been shared, the materials to replicate a study have been made available, and whether the study has been preregistered in a study registry.

In addition to the general meta-data discussed above, individual studies can be represented as Psych-DS compliant sub-folders to facilitate data reuse. The [Psych-DS](#) standard requires a `dataset_description.json` file, which is a machine-readable codebook describing the contents of a specific dataset, including every variable, its measurement scale, its value range, the originating data file, and potentially additional information such as descriptive statistics. Unlike `ro-crate-metadata.json`, which describes the provenance, contributions, and general properties of the data and code, this file describes what the data mean, enabling software to more accurately process variables in data files. Machine-readable data descriptions support findability, reuse and reproducible analyses. They also allow users to search databases for studies that collected specific data. The Psych-DS JSON-LD document is based on Schema.org's Dataset model, extended with Psych-DS specific fields. These JSON files can be generated automatically from well-organized repositories, if they contain an accurate codebook, and use sensible file and folder names.

Even more fine-grained schemas exists for subsets of datafiles, such as data of psychological scales. The [Open Scale Definition](#) (OSD) is a machine-readable format to share scales metadata that can be used for for questionnaires and rating instruments. It is part of the Psychology Experiment Building Language created for building and running psychological experiments ([Mueller & Piper, 2014](#)). An `.osd` file has two top-level parts: a definition (the structure, language-neutral) and translations (the actual text, keyed by language code, so the same structure can carry versions of a scale in different languages). The OSD specification requires `scale_info`, `likert_options`, `dimensions`, `items` and `scoring` (although an entry can be set to NULL). The scale info contains a name, code, and abbreviation, a citation to the original publication of the scale, license information, and other descriptive information. The JSON file can also contain information about the likert-scale response options, the dimensions of the scale (such as sub-scales, and which items belong to which subscale), how the scale is scored, and the specific text for each item, in different translations if available. FAIR scale meta-data facilitates re-use, but can also promote standardization in how

scales are used and scored across studies.

5 File Formats

Data should be stored in open source formats, such as a .csv file, or software specific files, such as .svp, .jasp or .omv. Very large data files should be stored in file formats that allow partial downloads of information through range requests (e.g., plain text in .csv instead of excel files in .xlsx). If data is collected with commercial software, or when data is not collected digitally (e.g., a questionnaire printed on paper), it is not always possible to make raw data (data in the form it is originally observed) available in open file formats. In such cases, the raw data files should be stored in a folder called “raw” inside the data folder (in a .zip archive if there are many files or if the files are large) for archival purposes, and exported as directly as possible to an open file format, such as csv. CSV files must be comma separated and follow the [RFC 4180 specification](#) and utf-8 encoded. Any commas used within the data in a csv file itself must be enclosed by double-quotes (for example, a row with an open text entry would be stored as pp1, 34, "To be, or not to be."). Do not use plain .txt files for data storage, as machines do not automatically know how to process .txt files. Do not use .rdata files to store R output (as these can contain multiple objects, and a machine does not automatically know how to process it), but use .rds files instead. Best practice is to ensure that the file extension accurately reflects the underlying file format. For example, do not store a JSON formatted file in a .csv or .txt file, but save it as a .json file.

It is possible to store data in Excel spreadsheets (.xls or .xlsx) as this is an Open Office XML format that can be read in any open source software. However, certain practices should be avoided. First, one should not store data in multiple tabs, as file names are used to classify data files, and when a spreadsheet contains multiple tabs, the name can't distinguish the data in each tab. Instead of using tabs, store multiple files. Second, data should be rectangular (each column contains the same number of rows) to be read in to other software. Spreadsheets with merged cells can't be read in as a data frame, and spreadsheets with empty rows or columns can lead to problems. Leave truly empty cells empty, as software can replace empty excel cells with an empty value (e.g., as NA in R). Avoid using a coding scheme for missing values, such as 999 = 'no response', 888 = 'hardware failure', but if you cannot avoid this (e.g., you don't

have control over the raw data), specify the coding scheme in the meta-data file. Do not rely on colour coding, comments, or hyperlinks to communicate information. This information is often lost when the data is read into other software, so always add such information in dedicated columns (e.g., a ‘notes’ column). CSV and Excel files should include headers in the first row, and only a single row should be used to label columns.

6 Codebooks and data dictionaries

A codebook, or data dictionary, describes data files. The two terms are used interchangeably, and a codebook and data dictionary largely contain the same information. Buchanan et al. (2021) note that a codebook customarily describes survey variables only, and can be made when data is collected, while a data dictionary is more commonly created in the planning stage, and additionally documents the file structure. Every data file (or set of data files with identical structure) needs a codebook. Data files should have a single-row header that contains column names, and the codebook explains what these names stand for and what the values in the columns mean. Some data file formats (e.g., SPSS, Stata, SAS, JASP, jamovi, Qualtrics) are labelled, and thus contain an embedded codebook.

A complete codebook should contain six pieces of information. First, it contains the column name, exactly as it appears in the data file (e.g., `panas-1`). Second, it contains a label that describes the column in prose (e.g., `positive and negative affect schedule item 1`). If data did not originate from a survey question, describe what the values in the column are (e.g., reaction times, correct or false responses, reading times, etc). The goal of the label is to provide a human-readable explanation of the variable, and add context. If the item is part of an existing scale, this is the place to mention the full name of the scale. A codebook at a minimum needs a column name and a label. Third, for data with a limited number of response options, provide the value labels. For scales, these are how the response options were labeled (e.g., 1 = very negative; 2 = somewhat negative; 3 = neutral; 4 = somewhat positive; 5 = very positive). If only the endpoints are labeled, then only enter those two values (e.g., 1 = very negative; 5 = very positive). Fourth, for data that was generated based on survey questions, enter the verbatim question text (Interested). Fifth, if it is not possible to specify a single coding scheme for missing data across all data columns in

the meta-data file of the repository (see above) then an individual column coding scheme for missing data can be included. Sixth, it can be useful to provide coding instructions in the codebook. One important coding instruction is `reverse scored` for items that should be reverse coded before combining into a scale mean. Other coding instructions can pertain to how the score will be used or has been used in the calculation of scales (e.g., `mean of PANAS items 1-10`) or that scores need to be transformed before analyzing. If coding instructions are complex in a way that doesn't lend itself to a tabular format, refer to a README or other document that explains the coding or, ideally, provides code that transforms the data into scores.

The easiest way to create a codebook is to use data analysis software, and complete all codebook related fields in these tools. If your data are in SPSS (`.sav`), Stata (`.dta`), SAS (`.sas7bdat`), JASP (`.jasp`), jamovi (`.omv`), or Qualtrics (`.qsfx`), fill in the variable and value labels inside that software and share the file. An important benefit of these file formats is that they combine the data and codebook in one file, which can be read without error. A limitation of these formats is that beyond `.sav` and `.jasp` files, the other formats do not allow users to specify missing values, and beyond Qualtrics, none of the formats allow specification of the exact item. Finally, none of these software packages have a dedicated field for coding instructions. Therefore, if any of these additional fields are needed, users still need to create a dedicated codebook. PsychDS provides a schema to describe datafiles in a JSON formatted codebook. However, many codebooks have a tabular structure, and the column headers specify the variable information that is coded. An example of a table is provided below.

variable-		missing-		coding-	
name	label	values	values	question	instructions
pid	Participant identifier				
age	Age in years		-99	What is your age in years?	

variable-			missing-		coding-
name	label	values	values	question	instructions
gender	Self-reported gender	1 = Man; 2 = Woman; 3 = Non-binary	99	Which best describes your gender?	
bds_1	Breakup Distress item 1	1 (Strongly disagree) to 5 (Strongly agree)	99	I feel overwhelmed by the breakup.	
bds_3	Breakup Distress item 3	1 (Strongly disagree) to 5 (Strongly agree)	99	I am coping well.	reverse scored: 6 - bds_3
bds_mean	Breakup Distress mean score				mean of bds_1 to bds_10 after reversing bds_3

Although tabular codebooks can be typed out in a .csv file, it might be most convenient to use a spreadsheet to create the codebook. When creating the codebook, a row of column names can be copied from the data file, and transposed (i.e., flipped to from one row to one column) and pasted in the spreadsheet. This should guarantee that the column names in the data file and codebook match perfectly. Make sure all values are stored as text – Excel might transform 1–5 into the date 5-Jan. The codebook can be stored as a .ods, .xlsx, or .csv file. The first two formats are easier to read by a human, while csv files are more interoperable. Do not store codebooks as word documents or pdf files, as the information in these documents can't be parsed reliably by a machine. Name the file codebook or data-dictionary. If there is only one data file in the same folder, there is no further need to use the filename to match the

codebook to a data file, as the folder groups related files. But if a folder has multiple data files, use the filename to match each codebook to its corresponding data file (e.g., `pilot-1a_data.csv` and `pilot-1a_codebook.csv`).

7 Tabular data files

Data files should be rectangular tables with a tidy structure, with one header row, one variable per column, and one observation per row. Analysis software such as SPSS, JASP and Jamovi will force these data formats on users, but when data files are created in spreadsheets it is very tempting to perform actions that deviate from this standard. It is common to see a header spanning multiple rows, empty columns or rows, or written notes at the bottom of a spreadsheet. Best practices for column names are more strict than best practices for file names, as only letters, digits, and underscores should be used (e.g., `participant_id`, `mean_reaction_time`) to make referencing columns in analysis code more straightforward. In R, referring to a column can be done with `df$participant_id`. If column names include a space or a dash, the column name needs quoting (e.g., `df$`participant-id`` or `df$`participant id``). Special characters often need to be escaped to print, or will be automatically renamed when importing the data file in other software, which could lead to identical column names if two columns only differ in a special character. This is also true if characters with diacritics are used. Keep column names shorter than 32 characters, as this is the limit for some analysis software packages. All column names should be unique within a data file, and columns that represent the same information in different files, such as `participant_id`, should have the same header if possible. Column names need to match the values in the codebook exactly.

Use column names that can be understood (e.g., do not use `SE1` but `self_esteem_scale_1`, and never use generic names like `VAR1`). Where possible, use column names that match the words used to refer to these data columns in the manuscript. If columns belong together use the same prefix (e.g., `self_esteem_scale`) and increasing numbers (e.g., `_1`, `_2`, `_3`). If columns correspond to a scale with subscales, the labeling convention discussed so far can be repeated for the subscale (e.g., `panas_1_positive_1` for the positive affect subscale of the positive and negative affect schedule). This will make it

easier for machines to automatically identify scales and subscales in datasets. Scale items should occupy adjacent positions in a dataframe. If multiple columns relate to the same response, such as the response value and the response time for a question, clearly label the properties that each column contains (e.g., `mood_1_response`, `mood_2_response`, `mood_1_reactiontime`, `mood_2_reactiontime`).

There should be no columns or rows with exclusively empty values. Blank cells are ambiguous as they do not distinguish between a deleted column, a column without recorded values, or a column that is mistakenly empty. If a column contains no values, replace the empty cells during data preprocessing with a value that indicates nothing was recorded (e.g., 'none' or 0) and add the meaning of the value in to the codebook. Column names and values in columns should not contain leading or trailing whitespace (e.g., not `yes` or `yes`, but `yes`). If columns are almost completely numeric, do not add a single text value (e.g., 'no response') as it prevents the entire column from being read as numeric values. Carefully check if the raw data contains personal information (e.g., email addresses, IP addresses) and anonymized the data file before sharing it publicly. Screen open text for identifying information.

Discussion

Data and code should be shared where possible to facilitate checking and reuse. Repositories containing FAIR data and code should allow machines to find and access all content. This requires data and code to be made available in online repositories that can feasibly be downloaded, where files and folders are clearly named to allow automatic categorization of files, the contents of the repository are described, files are shared in open formats, a codebook is included, data files are well-structured, and, where relevant, meta-data about measures and tasks is provided. Schema such as Psych-DS, RO-Crate JSON-LD, DDI-Lifecycle, CRediT, Open Scales Definition, and Behavense Data Model facilitate FAIR data and code.

Adhering to best practices for FAIR data and code is a complex and effortful task. There are over 100 recommendations for best practices in this document, and it is unreasonable to expect researchers or data managers to remember and implement them all. A solution to overcome limitations in memory and training is to automate the generation of FAIR

data and code as much as possible. Software solutions could be developed that scan a repository and assist researchers in iteratively improving how FAIR their data and code is. Metacheck (<https://github.com/scienceverse/metacheck>) is one implementation of this approach: it scans a repository, classifies its files, retrieves information from data files and codebooks, and can assist in creating meta-data files. Input from researchers is essential to provide all required meta-data, but this manual input should only be required where needed.

The potential of machine-readable data and code is immense. By scanning data repositories, all found data files can be processed automatically, and meta-data can be stored in a central database. It would then, for example, be easy to search the database for data files where researchers have administered specific scales or tasks, while filtering for demographic information about the sample. This would greatly facilitate the reuse of research data. Machine-readable code and data would also make automated reproducibility checks possible. This can increase the quality of scientific manuscripts, without increasing the burden on peer reviewers. Machine-readable data and code will likely play an important role in the ability of researchers to adhere to the principles in the code of conduct for research integrity to share FAIR data and code when possible.

As it is currently incredibly challenging to add metadata to data repositories, and as there are too many best practices in data sharing to remember them all, automated tools will play a big role in the adoption of FAIR data. If data and code can be processed by a machine, it becomes possible to validate the extent to which data and code is FAIR, and guide researchers interactively towards a data repository that is truly findable, accessible, interoperable, and re-usable.

References

- Arslan, R. C. (2019). How to Automatically Document Data With the codebook Package to Facilitate Data Reuse. *Advances in Methods and Practices in Psychological Science*, 2(2), 169–187. <https://doi.org/10.1177/2515245919838783>
- Buchanan, E. M., Crain, S. E., Cunningham, A. L., Johnson, H. R., Stash, H., Papadatou-Pastou, M., Isager, P. M., Carlsson, R., & Aczel, B. (2021). Getting Started Creating Data Dictionaries: How to Create a Shareable Data Set. *Advances in Methods and Practices in Psychological Science*, 4(1), 2515245920928007. <https://doi.org/10.1177/2515245920928007>
- Lakens, D., & DeBruine, L. M. (2021). Improving Transparency, Falsifiability, and Rigor by Making Hypothesis Tests Machine-Readable. *Advances in Methods and Practices in Psychological Science*, 4(2), 2515245920970949. <https://doi.org/10.1177/2515245920970949>
- Mueller, S. T., & Piper, B. J. (2014). The Psychology Experiment Building Language (PEBL) and PEBL Test Battery. *Journal of Neuroscience Methods*, 222, 250–259. <https://doi.org/10.1016/j.jneumeth.2013.10.024>